

Kubernetes Üzerinde OneUptime Dağıtımı

Helm, values.yaml Miras Mekanizması ve nodeSelector / nodeAffinity Kavramları

Bu doküman, "Kubernetes Üzerinde Dağıtık OneUptime İzleme Mimarisi" projesi kapsamında kullanılan üç temel kavramı — **Helm**, **values.yaml miras (inheritance)** mekanizması ve **nodeSelector / nodeAffinity** ayrımını — teknik detaylarıyla, projeden somut örneklerle açıklamaktadır.

1. Helm Nedir, Ne İşe Yarar?

Helm, Kubernetes için bir **paket yöneticisi** ve **şablon (template) motorudur**. Linux dünyasındaki `apt` veya `yum`'un Kubernetes karşılığı gibi düşünülebilir — ama sadece paket kurmakla kalmaz, karmaşık uygulamaların onlarca Kubernetes kaynağını (Deployment, Service, StatefulSet, ConfigMap, Secret, PersistentVolumeClaim...) tek bir komutla, parametrik olarak oluşturur.

1.1 — Helm olmadan ne olurdu?

OneUptime gibi bir sistem; bir uygulama sunucusu (app), bir ters proxy (nginx), bir veritabanı (postgresql), bir önbellek (redis), bir analytics veritabanı (clickhouse) ve birden fazla izleme ajanı (probe) içerir. Helm olmadan bu sistemi kurmak için:

- Her bileşen için ayrı ayrı `Deployment.yaml`, `Service.yaml`, `ConfigMap.yaml` dosyaları elle yazılırdı (muhtemelen 20-30 dosya).
- Farklı ortamlar (test, prod) için bu dosyaların kopyaları tutulur, aralarındaki farklar elle senkronize edilirdi.
- Bir değeri değiştirmek (örneğin bellek limiti) için doğru dosyayı bulup elle düzenlemek gerekirdi.

1.2 — Helm ile ne değişir?

Helm, bu manifestleri **chart** adı verilen bir pakette toplar. Bir chart üç ana bölümden oluşur:

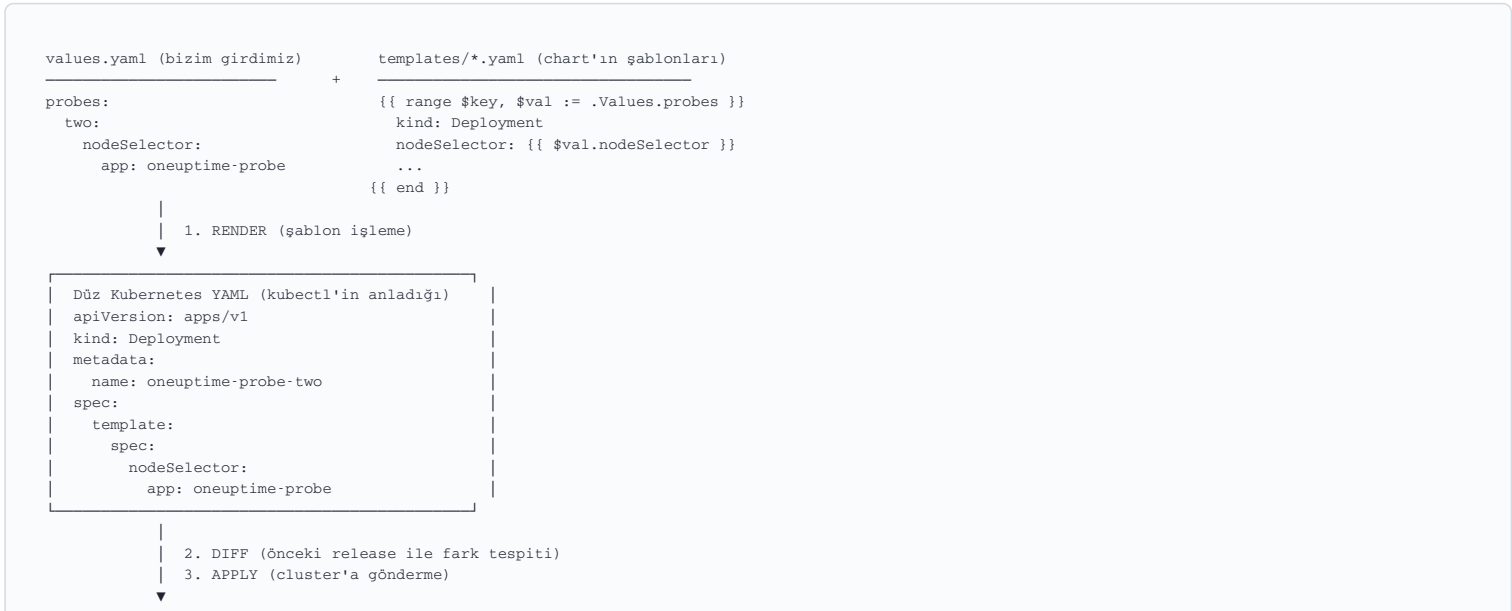
Bileşen	Görevi
<code>Chart.yaml</code>	Chart'ın adı, sürümü, bağımlılıkları (metadata)
<code>values.yaml</code>	Şablonlara enjekte edilecek varsayılan parametreler
<code>templates/</code>	Go template syntax'ı (<code>{{ }}</code>) içeren ham Kubernetes YAML şablonları

Bizim projemizde kullandığımız `oneuptime/oneuptime` chart'ının `templates/probe.yaml` dosyasında, örneğin, şöyle bir şablon parçası bulunur (basitleştirilmiş hâli):

```
# templates/probe.yaml (chart geliştiricisinin yazdığı şablon)
{{ range $key, $val := .Values.probes }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: probe-{{ $key }}
spec:
  template:
    spec:
      nodeSelector: {{ $val.nodeSelector | toYaml | nindent 6 }}
      containers:
        - image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
{{ end }}
```

Bu şablon, **bizim yazdığımız kod değil** — OneUptime ekibinin GitHub'da yayınladığı, herkesin kullanabildiği hazır bir şablon. Biz sadece `values.yaml` dosyasında `probes.two.nodeSelector` gibi değerler tanımlayarak bu şablona **parametre** veriyoruz.

1.3 — Helm komutu çalıştığında arka planda ne olur?



Bu üç adım (**render** → **diff** → **apply**), her `helm install` ve `helm upgrade` çalıştığında otomatik gerçekleşir. Projede sık kullandığımız şu komut, sadece render adımını gösterir ve cluster'a hiçbir şey göndermez:

```
helm template oneuptime oneuptime/oneuptime -f values.yaml -f probe2-values.yaml
```

Önemli: Helm ayrıca her `install` / `upgrade` işlemini bir "release revizyonu" olarak Kubernetes'in kendi içinde (bir Secret nesnesi olarak) saklar. Bu sayede `helm rollback oneuptime 1` gibi bir komutla önceki, çalışan sürüme geri dönebiliriz — projede yaşadığımız hatalı upgrade'ler sonrası bu özellik bir güvenlik ağı görevi görür.

1.4 — Diff mekanizması: neden sadece değişen pod'lar etkilenir?

Projede defalarca gözlemediğimiz bir davranış vardı: `probe2-values.yaml` dosyasını ekleyip `helm upgrade` yaptığımızda, sadece yeni bir `oneuptime-probe-two` Deployment'ı oluştu; `app`, `nginx`, `postgresql` gibi diğer bileşenlerin `RESTARTS` sayaçları değişmedi. Bunun sebebi, Helm'in render ettiği yeni manifest kümesini **önceki release'in manifestleriyle karşılaştırıp yalnızca farklı olan kaynakları güncellemesidir** — değişmeyen bir Deployment'ın pod'ları yeniden başlatılmaz.

2. values.yaml'daki Değerler Nasıl Miras Alınır (Inheritance)?

Bu, projede en çok karıştırılan ve en kritik konulardan biriydi. Helm chart'larında değerler **hiyerarşik olarak** tanımlanır ve **en spesifik (en derin) tanım, genel tanımı ezer (override eder)**.

2.1 — Global / Chart-wide varsayılan

Chart'ın `default-values.yaml` dosyasının en altına yakın bir yerde, hiçbir üst anahtarın altında olmayan (indent'siz) bir `nodeSelector:` tanımı bulunur:

```
# Chart'ın en genel (global) varsayılanı — herkes miras alabilir
nodeSelector:
```

Bu, varsayılan olarak **boş**tur (değer atanmamış). Biz projede bunu doldurduk:

```
nodeSelector:
  app: oneuptime-core
```

2.2 — Servis-bazlı (per-service) override

Chart içindeki her servis bloğunun (`app`, `nginx`, `worker`, `probes.one` vb.) **kendi**, boş bir `nodeSelector: {}` alanı vardır:

```
app:
  nodeSelector: {} # boş harita — kendi değeri yok

nginx:
  nodeSelector: {} # boş harita — kendi değeri yok
```

2.3 — Miras kuralı: boş bırakılan servisler, global değeri miras alır

Chart'ın şablon kodu (Helm'in `tpl` / `toYaml` fonksiyonları ve Go template mantığıyla) genelde şu örüntüyü izler:

```
{{ if $val.nodeSelector }}
  nodeSelector: {{ $val.nodeSelector | toYaml }}
{{ else }}
  nodeSelector: {{ .Values.nodeSelector | toYaml }} # global'e geri düş
{{ end }}
```

Yani: **eğer bir servisin kendi `nodeSelector` 'ı boşsa, o servis otomatik olarak global (chart-wide) `nodeSelector` değerini miras alır**. Eğer servis kendi değerini tanımlarsa, bu global değeri **tamamen ezer** (üzerine yazar, birleştirmmez).

2.4 — Projede bunu nasıl kullandık?

Projede `probes.one` (varsayılan probe) için hiç `nodeSelector` tanımlamadık — bilerek boş bıraktık, çünkü zaten global `nodeSelector: app: oneuptime-core` tanımlıydı ve `probes.one` 'ın Node 1'e gitmesini istiyorduk. Bu, miras mekanizmasından faydalanarak **daha az kod yazmamızı** sağladı:

```
# values.yaml — probes.one için AYRICA nodeSelector yazmadık,
# çünkü global nodeSelector zaten Node 1'i işaret ediyor.
nodeSelector:
  app: oneuptime-core

probes:
  one:
    key: "" # nodeSelector belirtilmedi → global'i miras alır
```

Ama `probes.two` için ise **bilinçli olarak kendi `nodeSelector` 'ını tanımladık** — çünkü onun global değerden **farklı** bir node'a (Node 2) gitmesini istiyorduk:

```
probes:
  two:
    nodeSelector:
      app: oneuptime-probe # kendi değeri global'i EZER
```

2.5 — Alt-chart'lar (subchart) miras zincirinin DIŞINDADIR

Projede karşılaştığımız önemli bir istisna: `postgresql`, `redis`, `clickhouse` gibi bileşenler chart içinde bağımsız **alt-chart (subchart)** olarak paketlenmiştir (genelde Bitnami'nin hazır chart'ları). Bu alt-chart'ların kendi `nodeSelector` alanları, üst (parent) chart'ın global `nodeSelector`'ından **miras almaz** — çünkü farklı bir "values scope"una aittirler. Bunu ilk kurulumda pratik olarak gördük: global `nodeSelector`'ı ayarladığımızda `postgresql`, `redis`, `clickhouse` pod'ları hâlâ Node 2'ye düşmüştü — bu yüzden her birine **ayrı ayrı, açıkça** `nodeSelector` tanımlamamız gerekti:

```
postgresql:
  primary:
    nodeSelector:
      app: oneuptime-core # global'den AYRI, elle tanımlanmalı

redis:
  master:
    nodeSelector:
      app: oneuptime-core
```

2.6 — Miras hiyerarşisinin özeti

Öncelik (yüksekten düşüğe)	Kaynak	Örnek
1 (en yüksek)	<code>helm upgrade -f dosya2.yaml</code> (sonradan verilen dosya)	<code>probe2-values.yaml</code>
2	<code>helm install/upgrade -f dosya1.yaml</code> (ilk verilen dosya)	<code>values.yaml</code>
3	Servis-bazlı chart varsayılanı	<code>app.nodeSelector: {}</code>
4 (en düşük)	Global chart-wide varsayılanı	<code>nodeSelector:</code> (kök seviye)

Not: Birden fazla `-f` dosyası verildiğinde (`helm upgrade oneuptime oneuptime/oneuptime -f values.yaml -f probe2-values.yaml`), **sonraki dosya öncekini ezer** — birleştirme sadece farklı anahtarlarda olur, aynı anahtar için son değer kazanır. Bu yüzden projede sürekli "her iki dosyayı birlikte verin, yoksa biri sıfırlanır" uyarısını tekrarladık.

3. nodeSelector ve nodeAffinity Arasındaki Farklar

3.1 — Neden Kubernetes'te böyle bir mekanizmaya ihtiyaç var?

Normalde Kubernetes'in **scheduler**'ı (zamanlayıcı), bir pod'u hangi node'a yerleştireceğine **otomatik** karar verir — kaynak uygunluğuna (boş CPU/RAM) bakar. Ama bazı durumlarda bu kararı biz kısıtlamak isteriz:

- Veritabanı pod'unu SSD diskli bir node'a sabitlemek
- GPU gerektiren bir işi sadece GPU'lu node'lara göndermek
- Bizim projemizde:** iş yükünü izole etmek — core bileşenleri Node 1'e, probe'u Node 2'ye zorlamak

Bunu sağlayan iki mekanizma vardır: **nodeSelector** (basit, eski) ve **nodeAffinity** (gelişmiş, esnek).

3.2 — nodeSelector: Basit Eşitlik Filtresi

`nodeSelector`, pod şablonunun (`spec.nodeSelector`) altına yazılan, **key=value eşleşmesi arayan** en basit mekanizmadır:

```
spec:
  nodeSelector:
    app: oneuptime-core
```

Bu satır şu anlama gelir: **"Bu pod'u SADECE `app=oneuptime-core` etiketine sahip bir node'a yerleştir; böyle bir node yoksa, pod `Pending` durumunda kalsın (asla başka yere gitme)."**

Özellik	Değer
Söz dizimi	Basit key-value çifti
Desteklenen operatör	Sadece eşitlik (<code>=</code>)
Birden fazla koşul	Mümkün (birden fazla key-value), ama hepsi VE (AND) ile bağlanır
Esneklik	Yok — ya tam eşleşir ya da pod hiç çalışmaz
"Tercih" kavramı	Yok

3.3 — nodeAffinity: Gelişmiş, Kural Tabanlı Yerleştirme

`nodeAffinity`, pod'un `spec.affinity.nodeAffinity` altında tanımlanır ve **iki farklı mod** sunar:

a) requiredDuringSchedulingIgnoredDuringExecution — Zorunlu kural

`nodeSelector` ile **işlevsel olarak birebir aynı sonucu** verir, ama daha uzun ve daha zengin bir syntax'la:

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: app
                operator: In
                values:
                  - oneuptime-core
```

b) preferredDuringSchedulingIgnoredDuringExecution — Tercih (soft) kural

Bu, `nodeSelector`'da **hiç bulunmayan** bir özelliktir:

```
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          preference:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - oneuptime-core
```

Bu, şu anlama gelir: **"Tercihen `app=oneuptime-core` etiketli node'a git, ama uygun node yoksa, pod'un hiç çalışmamaktansa başka bir node'a gitmesi daha iyidir."** Scheduler, birden fazla `preferred` kural varsa, `weight` (ağırlık) değerlerine göre en uygun node'u puanlayarak seçer.

3.4 — nodeAffinity'nin desteklediği zengin operatörler

Operatör	Anlamı	Örnek kullanım senaryosu
<code>In</code>	Değer, listedeki değerlerden biriye eşleşir	<code>app in (oneuptime-core, oneuptime-backup)</code>
<code>NotIn</code>	Değer, listedeki değerlerden hiçbiri değilse eşleşir	<code>app notin (oneuptime-probe)</code> — "probe node'una asla gitme"

Exists	Sadece key'in var olup olmadığına bakar, değere bakmaz	gpu exists — "üzerinde GPU etiketi olan herhangi bir node"
DoesNotExist	Key'in olmadığı node'ları seçer	maintenance doesnotexist
Gt / Lt	Sayısal karşılaştırma (büyüktür/küçüktür)	disk-size-gb Gt 100

nodeSelector bu operatörlerin **hiçbirini** desteklemez — sadece düz eşitlik yapabilir.

3.5 — İki mekanizmanın karşılaştırmalı özeti

Özellik	nodeSelector	nodeAffinity
Söz dizimi karmaşıklığı	Basit	Daha uzun, daha karmaşık
Zorunlu (hard) kural	✔ Var (tek modu bu)	✔ Var (required...)
Tercihli (soft) kural	✘ Yok	✔ Var (preferred...)
Operatör çeşitliliği	Sadece eşitlik	In / NotIn / Exists / DoesNotExist / Gt / Lt
Ağırlıklandırma (weight)	✘ Yok	✔ Var (preferred modda)
Kubernetes'teki durumu	Eski, basit; hâlâ desteklenir ama "legacy" sayılır	Güncel, önerilen (recommended) yaklaşım
Öğrenme eğrisi	Dakikalar içinde öğrenilir	Biraz zaman gerektirir

4. Projemizde Hangi Mekanizma Kullanıldı ve Neden?

Bu projede **nodeSelector** tercih edildi, çünkü ihtiyacımız **kesin ve zorunlu bir yerleştirmeydi**: "core bileşenler **sadece** Node 1'de, probe **sadece** Node 2'de çalışsın" — burada hiçbir esnekliğe (tercihe) ihtiyaç yoktu. Bu tür net izolasyon senaryolarında nodeSelector, nodeAffinity'nin **required** modunun aynı sonucunu **daha az kod yazarak** verir.

nodeAffinity'yi ne zaman tercih ederdik? Eğer amacımız katı izolasyon değil, **yüksek erişilebilirlik (HA)** olsaydı — örneğin "core bileşenler tercihen Node 1'e gitsin, ama Node 1 doluysa veya arızalıysa Node 2'ye de gidebilsin (hiç çalışmamaktansa)" — o zaman **preferredDuringSchedulingIgnoredDuringExecution** kullanmamız gerekirdi. Bu, projenin "çapraz izleme" felsefesiyle (sistemin bir kısmı çökse bile diğer kısmın haberdar olması) aslında örtüşen bir yaklaşımdır; ancak görevin "hangi pod'un hangi node'da olduğunu net kanıtlama" gereksinimi (**kubectl get pods -o wide** ile doğrulama) için nodeSelector'ın katılığı daha doğru ve sınanabilir bir seçimdi.

4.1 — Projede kullanılan gerçek values.yaml örneği

```
nodeSelector:
  app: oneuptime-core # global varsayılan → Node 1

app:
  nodeSelector:
    app: oneuptime-core # açıkça belirtildi (subchart olmasa da netlik için)

postgresql:
  primary:
    nodeSelector:
      app: oneuptime-core # subchart → miras almaz, elle yazıldı

probes:
  one:
    key: "" # nodeSelector yazılmadı → global'i miras alır (Node 1)
  two:
    nodeSelector:
      app: oneuptime-probe # global'i EZER → Node 2
```

4.2 — Sonuç: Doğrulama komutu ve gerçek çıktı

```
kubectl get pods -n oneuptime -o wide
```

NAME	READY	STATUS	NODE	
oneuptime-app-...	1/1	Running	minikube	# Node 1
oneuptime-nginx-...	1/1	Running	minikube	# Node 1
oneuptime-postgresql-0	1/1	Running	minikube	# Node 1
oneuptime-probe-one-...	1/1	Running	minikube	# Node 1 (miras alındı)
oneuptime-probe-two-...	1/1	Running	minikube-m02	# Node 2 (override edildi)

Bu çıktı, hem miras mekanizmasının (probe-one, global değeri hiç yazmadan Node 1'e gitti) hem de override mekanizmasının (probe-two, kendi tanımıyla Node 2'ye gitti) doğru çalıştığının somut kanıtıdır.